

Tomato Leaf Disease Classification Using RepViT and MobileNetV4 for Mobile Application

1st Fa'iq Haidar Prasetya
Computer Science Department
School of Computer Science
Bina Nusantara University
Bandung, Indonesia
faiq_prasetya@binus.ac.id

2nd Nicholas Nathaniel Sugiarto
Computer Science Department
School of Computer Science
Bina Nusantara University
Bandung, Indonesia
nicholas.sugiarto@binus.ac.id

3rd Stefany Go
Computer Science Department
School of Computer Science
Bina Nusantara University
Bandung, Indonesia
stefamy.go@binus.ac.id

4th Abdul Harris Rangkuti
Computer Science Department
School of Computer Science
Bina Nusantara University
Bandung, Indonesia
rangku2000@binus.ac.id

Abstract— Tomato leaf diseases can negatively affect crop productivity and quality, making early disease identification an important task in agricultural management. Manual identification requires visual inspection and may be difficult when different diseases exhibit similar symptoms. This study develops a tomato leaf disease classification system using deep learning models integrated into an Android-based mobile application. The study focuses on two classification architectures, namely RepViT and MobileNetV4. The models were trained using a Tomato Leaf Disease Dataset obtained from Roboflow and evaluated on ten tomato leaf classes. The experimental dataset was configured using a 70:20:10 training, validation, and testing scheme. Images were resized to 224×224 pixels and processed using normalization and augmentation techniques. Both models were trained using transfer learning with pretrained weights. The experimental results show that RepViT achieved an accuracy of 98.39%, a macro-averaged precision of 98.39%, recall of 98.44%, and F1-score of 98.40%. MobileNetV4 achieved an accuracy of 96.67%, precision of 96.63%, a macro-averaged recall of 96.74%, and F1-score of 96.66%. RepViT used 9.67 million parameters and required 24.88 ms of average inference time, while MobileNetV4 used 8.45 million parameters and required 8.87 ms. The results indicate that RepViT provides higher classification performance, while MobileNetV4 provides a smaller model and faster inference suitable for resource-constrained mobile applications. The developed application demonstrates the feasibility of integrating tomato leaf disease classification models into a mobile-based system.

Keywords— Tomato leaf diseases, Deep Learning, Repvit, MobileNetV4, Mobile Application.

I. INTRODUCTION

Plant diseases can significantly affect agricultural productivity and food production. [1] Tomato is an important horticultural crop that contributes to agricultural production and food supply in Indonesia. [2] Tomato plants are susceptible to various diseases caused by pathogens and pests. [3] Disease symptoms commonly appear on leaves

through changes such as discoloration, lesions, necrosis, and deformation. [4] Similar visual symptoms among different diseases can make manual disease identification difficult. [5] Early identification of plant diseases can support more timely disease management. [6] Computer vision and deep learning have therefore been applied to automate disease identification from plant images. [7] Image-based deep learning systems can learn visual characteristics from leaf images and classify disease categories without relying entirely on manual visual inspection. [8]. Mobile deployment introduces additional requirements because classification models must operate within limited computational and memory resources. MobileNetV4 was designed as a mobile-oriented architecture with efficient components for deployment across mobile processors and accelerators. [9] RepViT was developed by incorporating efficient architectural principles from lightweight vision transformers into a mobile-friendly convolutional architecture. [10]

The original study proposed a comparison involving several architectures and both classification and detection tasks. However, the available experimental results only contain classification experiments using RepViT and MobileNetV4. Therefore, this revised study limits its scope to the comparison of RepViT and MobileNetV4 for tomato leaf disease classification. The objective of this study is to evaluate the classification performance and computational characteristics of RepViT and MobileNetV4 using the same tomato leaf dataset. The evaluation considers accuracy, precision, recall, F1-score, confusion matrix, parameter count, model size, and average inference time. The study also integrates the trained classification models into an Android-based prototype that allows users to capture or select tomato leaf images for disease classification.

II. LITERATURE REVIEW

2.1 Tomato Leaf Disease Classification

Tomato leaf diseases include bacterial, fungal, viral, and test-related conditions that can affect plant health and crop productivity. [3] Common visual symptoms include leaf

spots, discoloration, curling, lesions, and tissue damage. [4] Deep learning has been widely investigated for plant disease recognition because neural networks can automatically learn discriminative features from images. [7] Previous research demonstrated that deep learning models can classify plant diseases using leaf images. [11]. Tomato disease classification has also been investigated using pretrained deep learning models. [12] Transfer learning can reduce the amount of training required when applying pretrained visual representations to plant disease datasets. [13]

2.2 RepViT

RepViT is a lightweight convolutional architecture developed from design principles inspired by Vision Transformers. [10] The architecture is designed to provide a favorable balance between classification accuracy and inference latency for mobile-oriented applications. [10]. In this study, the selected model is `repvit_m3.dist_in1k` from the `timm` model library. The model uses ImageNet-pretrained weights and receives input images with a spatial resolution of 224×224 pixels.

2.3 MobileNetV4

MobileNetV4 is a family of models designed specifically for efficient deployment in mobile environments. [9] The architecture introduces the Universal Inverted Bottleneck and Mobile MQA components to improve efficiency across mobile hardware platforms. [9]. MobileNetV4 was selected in this study because its mobile-oriented design provides an appropriate candidate for comparison with RepViT in a mobile disease-classification application.

2.4 Data Augmentation

Data augmentation can increase variation in training images and reduce the tendency of a model to overfit the training dataset. [14] Augmentation techniques can include geometric transformations and changes in image appearance.

In this study, the dataset contains preprocessing and augmentation operations provided through the Roboflow dataset configuration. Additional mild augmentation was also applied during model training.

III. METHODOLOGY

3.1 Research Methodology

The research consists of dataset preparation, preprocessing, model training, model evaluation, and mobile application integration. The study does not perform object detection or disease localization because the available experiments are formulated as image classification tasks. Two deep learning models, RepViT and MobileNetV4, are independently trained using the same dataset configuration. The models are subsequently evaluated using classification and computational metrics. The model results are then integrated into an Android-based prototype for tomato leaf disease classification.

3.2 Dataset

The dataset used in this study is the Tomato Leaf Disease Dataset, Roboflow Project Version 6. The dataset is available through the Roboflow project used in this research. [15]

The Roboflow dataset interface reports 8,611 total images. The experiment configuration uses a 70:20:10 training, validation, and testing arrangement. The exported experiment folders contain 6,051 training images, 1,696 validation images, and 870 testing images.

The retained classification results contain ten evaluated classes. The ten classes are Bacterial Spot, Early Blight, Late Blight, Leaf Mold, Septoria Leaf Spot, Spider Mites, Target Spot, Tomato Yellow Leaf Curl Virus, Tomato Mosaic Virus, and Healthy.

The retained training class-count output contains 6,045 class-labeled images, while the folder-level training configuration reports 6,051 training instances. Because class-level validation counts were not retained in the available experimental records, the validation set is reported only as an aggregate of 1,696 image instances.

3.3 Data Preprocessing

The Roboflow dataset configuration resizes images to 224×224 pixels. The dataset configuration also applies automatic orientation processing. The Roboflow augmentation configuration includes horizontal and vertical flipping, cropping with up to 20% maximum zoom, horizontal and vertical shearing up to 10° , brightness variation between -15% and $+15\%$, exposure variation between -10% and $+10\%$, and pixel noise up to 0.1%. For RepViT, additional training transformations include resizing to 224×224 pixels, horizontal flipping, vertical flipping, rotation up to 8° , and mild color jitter. The RepViT input images are normalized using the mean and standard deviation resolved from the selected pretrained model configuration. The evaluation transformation does not apply random augmentation. For MobileNetV4, the training configuration uses an input size of 224×224 pixels.

3.4 Model Training Configuration

Both models were trained using transfer learning with pretrained weights. The experiments were conducted using a CUDA-enabled environment with an NVIDIA Tesla T4 GPU. The CUDA version recorded in the training environment was 12.6. The random seed used for the experiments was 42. The batch size for both experiments was 32. The maximum number of training epochs was 30. The RepViT experiment used the `repvit_m3.dist_in1k` architecture with ImageNet-pretrained weights. The MobileNetV4 experiment used pretrained weights with label smoothing of 0.05. Early stopping was applied during both training experiments. RepViT achieved its best validation macro F1-score of 0.9882 at epoch 22. The RepViT training process continued until epoch 30 before early stopping was triggered. The total RepViT training time was 16.95 minutes. MobileNetV4 achieved its best validation macro F1-score at epoch 11. The MobileNetV4 training log reported early stopping at epoch 19.

3.5 Performance Evaluation

The models were evaluated using accuracy, precision, recall, and F1-score. Precision measures the proportion of predicted samples belonging to a class that are correctly classified. Recall measures the proportion of actual samples of a class that are correctly identified. F1-score provides a combined measure based on precision and recall. The macro

average calculates the arithmetic mean of the metric values across all evaluated classes. A confusion matrix was generated to observe the distribution of correct and incorrect predictions among the ten classes. Model efficiency was evaluated using parameter count, checkpoint size, RAM/VRAM model size, and average inference time. The reported inference time represents the model inference benchmark available in the experiment records.

3.6 Mobile Application

The classification models were integrated into an Android-based prototype. The application provides an image capture function and a gallery-selection function. The application processes the selected image and displays the

predicted tomato leaf disease class. The application was tested using an Android Studio Pixel 7 emulator. The emulator uses Android API Level 35. The virtual device has a display resolution of 1080×2400 pixels and a configured density of 420 dpi. The emulator configuration specifies four virtual CPU cores and 2 GB of RAM. The emulator uses an x86_64 system image. The reported model inference measurements were not recorded specifically as Pixel 7 emulator latency measurements. Therefore, the inference-time comparison is presented as a model-level computational benchmark rather than a physical smartphone performance measurement. Quantitative usability, CPU utilization, and memory-consumption results for the application were not retained in the available experimental records.

3.7 Flowchart

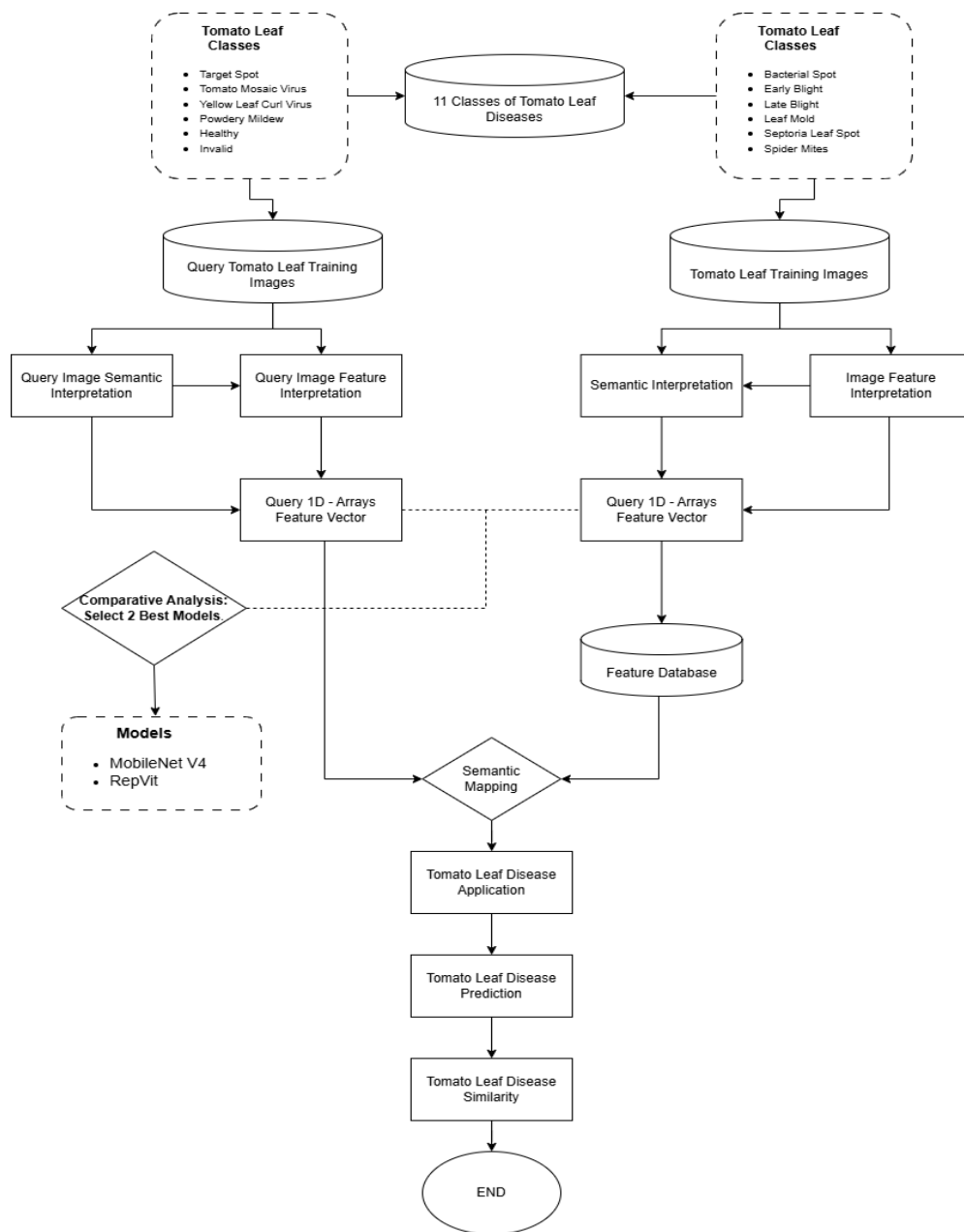


Fig. 3.1 Flow Diagram of Tomato leaf Image Retrieval

The process begins with the identification of 11 Classes of Tomato Leaf Diseases, which are stored as the primary reference database. The system handles two parallel streams: Query Tomato Leaf Training Images for user input and Tomato Leaf Training Images for the established dataset. Both streams undergo Semantic Interpretation and Image Feature Interpretation to analyze the context and visual characteristics of the tomato leaves. Following this, the Query 1D-Arrays Feature Vector process converts the interpreted data into a numerical format suitable for machine comparison.

A critical stage in this software engineering approach is the Comparative Analysis, where the system is designed to Select 2 Best Models, specifically MobileNet V4 and RepViT, due to their balance of accuracy and performance. The extracted features are then archived in a Feature Database for efficient retrieval. Through Semantic Mapping, the query vector is compared against the database using a Similarity Measure Model to find the closest match. To ensure the application remains responsive on handheld hardware, the system undergoes Mobile Optimization to reduce the model size and computational load. The process concludes with the Implementation & Output stage, where the diagnostic results and treatment recommendations are displayed to the farmer, followed by the END of the process.

IV. RESULT AND DISCUSSION

A. RepViT

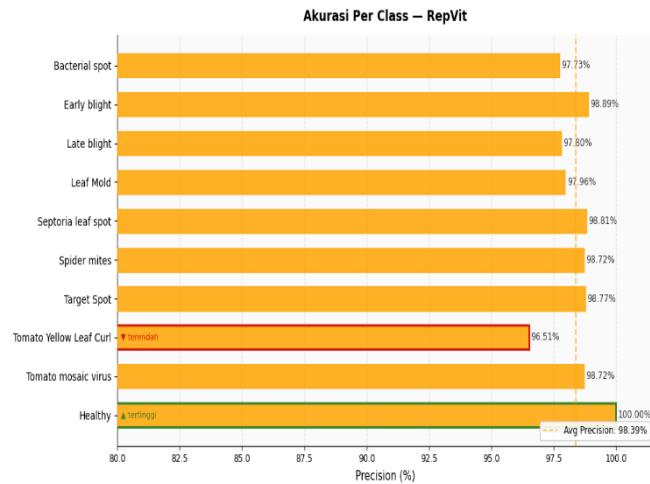


Fig 4.1 RepViT Classification Performance

Figure 4.1 presents the precision performance of the RepViT model across different tomato leaf disease classes. The model achieved an average precision of **98.39%**, indicating a high capability in correctly identifying disease categories while minimizing false-positive predictions. The highest precision was obtained in the **Healthy** class with **100.00%**, demonstrating that the model was highly effective in distinguishing healthy leaves from diseased ones. In contrast, the lowest precision was recorded for the **Tomato Yellow Leaf Curl Virus** class at **95.51%**, suggesting that this class shares visual similarities with other disease categories and is therefore more challenging to classify accurately. Overall, the results indicate that RepViT provides stable and reliable

precision performance across all classes, making it a robust model for tomato leaf disease classification applications.

B. MobileNetV4

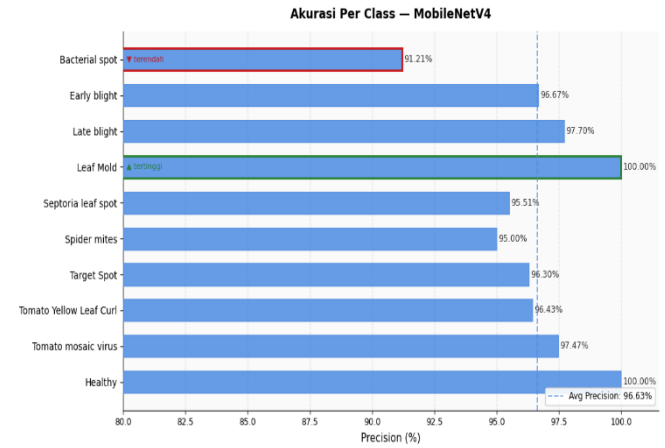


Figure 4.2 MobileNetV4 Classification Performance

Figure 4.2 illustrates the precision performance of the MobileNetV4 model for each tomato leaf disease class. The model achieved an average precision of **96.63%**, demonstrating its capability to classify tomato leaf diseases with a high level of correctness while maintaining computational efficiency. The highest precision was obtained in the **Leaf Mold** and **Healthy** classes, both achieving **100.00%**, indicating that all predictions for these classes were correctly identified without false-positive classifications. Conversely, the **Bacterial Spot** class recorded the lowest precision of **91.21%**, suggesting that some samples from other disease classes were incorrectly predicted as Bacterial Spot due to similarities in visual symptoms. Despite variations across classes, MobileNetV4 maintained consistently high precision values above 90% for all categories, highlighting its effectiveness as a lightweight deep learning model suitable for real-time tomato leaf disease classification on mobile and edge devices.

C. Comparison of Classification Performance

RepViT achieved higher values than MobileNetV4 across accuracy, macro precision, macro recall, and macro F1-score. The difference in accuracy between RepViT and MobileNetV4 was **1.72** percentage points. The difference in macro F1-score between the two models was **1.74** percentage points. Based on classification performance, **RepViT** was the best-performing model in this experiment.

D. Computational Efficiency

MobileNetV4 contains approximately 1.22 million fewer parameters than RepViT. The MobileNetV4 checkpoint is approximately **79% smaller** than the **RepViT** checkpoint based on the reported checkpoint sizes. MobileNetV4 also achieved an average inference time of **8.87 ms**, compared with **24.88 ms** for RepViT. The measured inference time of MobileNetV4 is approximately **2.8 times faster** than RepViT under the reported model-level benchmark.

These results provide quantitative support for selecting MobileNetV4 when computational efficiency and model size are prioritized. RepViT remains preferable when classification performance is prioritized.

E. Training Performance

RepViT reached its best validation macro F1-score of **98.82%** at epoch 22. The final training log shows that RepViT reached a training accuracy of approximately 99.98% at epoch 30. The validation accuracy at epoch 30 was approximately **98.76%**. The RepViT training process required 16.95 minutes.

MobileNetV4 reached its best recorded validation macro F1-score of approximately **96.96%** at epoch 11. The MobileNetV4 training log triggered early stopping at epoch 19. The training logs indicate that both models achieved high training and validation performance.

F. Mobile Application

The developed Android application provides two main image input options, namely taking a photograph and selecting an image from the gallery. The application then sends the selected image through the classification workflow and displays the predicted disease category. The application includes model information to distinguish the available classification models. The prototype was executed using an Android Studio Pixel 7 emulator. The emulator was configured with Android API Level 35, four virtual CPU cores, and 2 GB of RAM. The mobile prototype demonstrates that the trained models can be incorporated into an Android-based tomato leaf disease classification workflow. However, the available experimental records do not contain quantitative usability scores or CPU and memory measurements collected during application execution. Therefore, this study does not make a quantitative claim regarding user usability or physical smartphone resource consumption.

G. Mobile Application Simulation

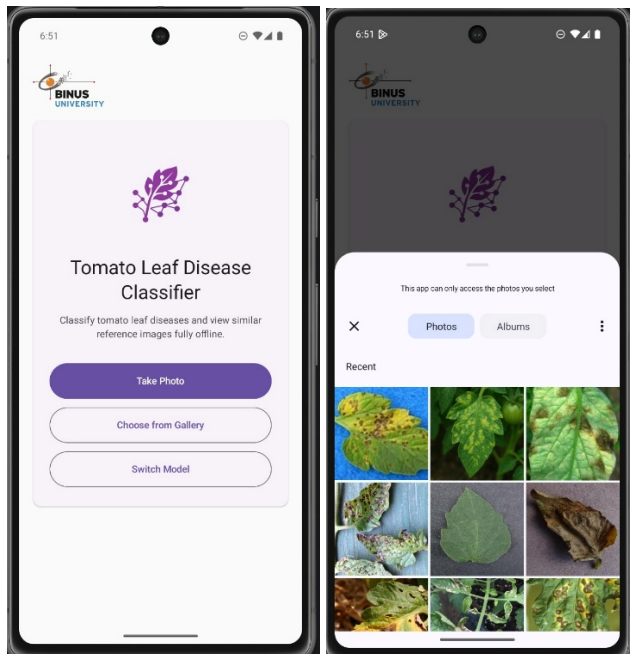


Fig. 4.3. Mobile application interface for tomato leaf disease classification: (a) home page and (b) tomato leaf image input.

Fig 4.3 (a) Home Page

This interface serves as the initial stage of the classification workflow. After an image source is selected,

the application obtains the corresponding tomato leaf image and forwards it to the classification process.

Fig. 4.3 (b) Tomato Leaf Image Input

The image input interface allows users to select a tomato leaf image from the device gallery. The selected image is used as the input for the disease classification model. This stage demonstrates the integration between the mobile application's user interface and the image classification pipeline.

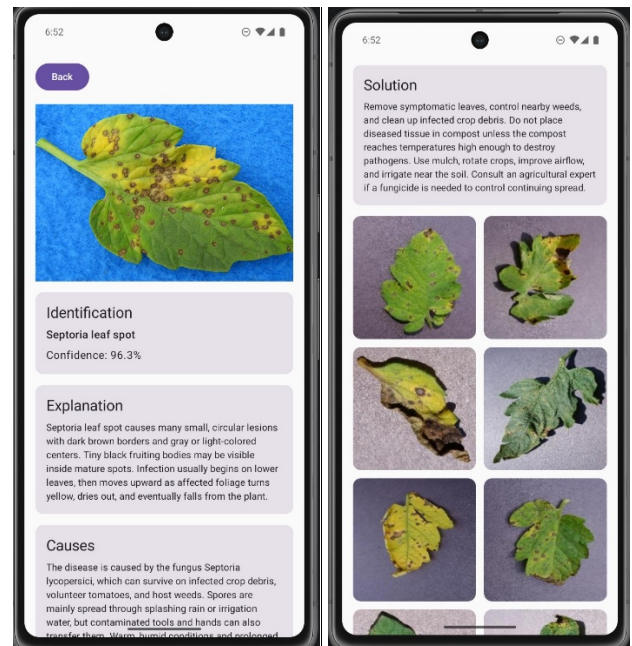


Fig. 4.4. Mobile application disease information: (a) classification result and (b) disease solution with reference images.

Fig. 4.4 (a) Classification Result

The classification result page displays the input tomato leaf image together with the predicted disease class and confidence score. In the presented example, the application identifies the leaf as **Septoria Leaf Spot** with a confidence score of **96.3%**. The page also provides an explanation of the identified disease, including its visual characteristics and potential causes. This information allows users to understand the basis of the classification result and the characteristics associated with the detected disease.

Fig. 4.4 (b) Disease Solution and Reference Images

The disease solution page provides information related to the management of the identified tomato leaf disease. It includes recommended control measures and several reference images showing tomato leaves affected by the corresponding disease. These visual references help users compare the detected symptoms with other examples and provide additional information for understanding the identified disease.

V. Conclusion

This study developed a tomato leaf disease classification system using RepViT and MobileNetV4 and evaluated both models based on accuracy, precision, recall, F1-score, confusion matrix, parameter count, model size, and inference time. RepViT achieved the best classification performance with 98.39% accuracy, 98.39% macro precision, 98.44% macro recall, and 98.40% macro F1-score. MobileNetV4 achieved 96.67% accuracy, 96.63% macro precision, 96.74% macro recall, and 96.66% macro F1-score. Although RepViT provided higher classification performance, MobileNetV4 required fewer parameters, with 8.45 million parameters compared with 9.67 million parameters for RepViT. MobileNetV4 also achieved a lower average inference time of 8.87 ms, compared with 24.88 ms for RepViT, indicating its advantage when model size and inference speed are prioritized for mobile applications.

The developed Android prototype demonstrates the feasibility of integrating tomato leaf disease classification into a mobile-based workflow. However, the evaluation was limited to the available test dataset and did not include independent field images or quantitative application-level usability and resource-consumption measurements. Therefore, the results should not yet be interpreted as comprehensive validation for real-world agricultural deployment. Future research should evaluate the models using independent field images and conduct direct on-device benchmarking on physical smartphones to further assess their practical deployment performance. application-level usability and resource-consumption measurements.

REFERENCE

- [1] Food and Agriculture Organization, The State of Food and Agriculture 2022, Rome, Italy: FAO, 2022.
- [2] Badan Pusat Statistik, Statistik Hortikultura Indonesia 2023, Jakarta, Indonesia: BPS, 2023.
- [3] S. Panno, S. Davino, G. Caruso, A. Bertacca, and G. Marchese, "A review of the most common and economically important diseases that undermine the cultivation of tomato crop in the Mediterranean basin," *Agronomy*, vol. 11, no. 11, 2021.
- [4] J. G. A. Barbedo, "A review on the main challenges in automatic plant disease identification based on visible range images," *Biosystems Engineering*, vol. 144, pp. 52–60, 2016.
- [5] A. K. Rangarajan, R. Purushothaman, and A. Ramesh, "Tomato crop disease classification using pre-trained deep learning algorithm," *Procedia Computer Science*, vol. 133, pp. 1040–1047, 2018.
- [6] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.
- [7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, Art. no. 1419, 2016.
- [8] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [9] D. Qin, C. Leichner, M. Delakis, M. Fornoni, S. Luo, F. Yang, W. Wang, C. Banbury, C. Ye, B. Akin, V. Aggarwal, T. Zhu, D. Moro, and A. Howard, "MobileNetV4: Universal models for the mobile ecosystem," in *Computer Vision – ECCV 2024*, 2024, pp. 78–96.
- [10] A. Wang, H. Chen, Z. Lin, J. Han, and G. Ding, "RepViT: Revisiting mobile CNN from ViT perspective," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15909–15920.
- [11] M. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep learning for tomato diseases: Classification and symptoms visualization," *Applied Artificial Intelligence*, vol. 31, no. 4, pp. 299–315, 2017.

- [12] R. Jelali, "Deep learning-based tomato leaf disease classification using transfer learning," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 4, pp. 233–240, 2022.
- [13] E. C. Too, Y. Li, S. Njuki, and Y. Liu, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.
- [14] S. Wagle, P. Khatiwada, and A. Shrestha, "Data augmentation techniques for plant disease recognition using deep learning," *Computers and Electronics in Agriculture*, vol. 190, 2021.
- [15] Roboflow, "Tomato Leaf Disease Dataset, Version 6," Roboflow. [Online]. Available: Tomato Leaf Disease Dataset [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

AUTHOR CONTRIBUTION

This research was prepared, supervised, and designed including proposing the model and algorithm used and writing of the paper by Abdul Haris Rangkuti. The experiment and collecting data of the tomato disease plant was carried out by Fa'iq Haidar Prasetya, Nicholas Axel Nathaniel Sugiarto, Stefany Go. All authors read and approved the manuscript.

DATA AVAILABILITY

An overview of the availability of Tomato Leaf Disease Classification datasets for the image classification can be found at link <https://app.roboflow.com/aol-ai-2nwdp/tomato-leaf-disease-q4df7-3jjdy/6>